Inferencia verificable en modelos de Aprendizaje Automático mediante Pruebas de Conocimiento Cero

Criptografía

Álvaro Martínez Alfaro

alvaro.martinez39@alu.uclm.es

7 de mayo de 2026

Escuela Superior de Ingeniería Informática, Albacete, España

Resumen

El aprendizaje automático se integra cada vez más en aplicaciones críticas bajo un modelo de inferencia como servicio en el que el proveedor no revela los detalles internos de su modelo. Esto introduce un problema de confianza: el usuario no puede verificar que la inferencia se ha realizado correctamente ni que el modelo empleado es el declarado. Las pruebas de conocimiento cero permiten demostrar la correcta ejecución de un cómputo sin revelar información secreta, ofreciendo una solución teórica al problema. Sin embargo, su aplicación a modelos de aprendizaje automático, campo conocido como *Zero-Knowledge Machine Learning*, presenta importantes desafíos técnicos derivados de la incompatibilidad entre la aritmética de punto flotante empleada por muchos modelos de aprendizaje automático y los cuerpos finitos sobre los que operan los protocolos criptográficos.

En este trabajo se estudian los fundamentos teóricos de ambos dominios y se analizan los principales obstáculos que surgen al combinarlos: la aritmetización de la inferencia mediante sistemas R1CS y programas aritméticos cuadráticos (QAP), la cuantización de pesos y activaciones a aritmética de cuerpo finito y el manejo de funciones de activación no lineales como ReLU mediante descomposición en *bits* y tablas de búsqueda. Se revisan brevemente algunos *frameworks* modernos, entre ellos *EZKL*, *Circom*, *ZEN* y *zkLLM* y, por último, se presenta una implementación en `MATLAB` como prueba de concepto que integra estas técnicas de forma didáctica.

Índice general

1 Introducción **1**

2 Fundamentos teóricos **2**

2.1 Una aproximación al aprendizaje automático 2

2.1.1 Redes neuronales artificiales, aprendizaje e inferencia 2

2.2 Pruebas de conocimiento cero 3

2.2.1 Pruebas interactivas 4

2.2.2 Propiedad de conocimiento cero 5

2.3 Pruebas de conocimiento cero para modelos de aprendizaje automático 6

2.3.1 SNARKs y aritmetización 7

2.3.2 Compromisos polinómicos y *blinding* 8

2.3.3 Cuantización y aritmética en $\mathbb{F}_p$ 9

2.3.4 Lidiando con la no linealidad 10

2.3.5 *Frameworks* y herramientas modernas 11

3 Prueba de concepto en *MATLAB* **12**

3.1 Cuantización y aritmética modular 13

3.2 Aritmetización en R1CS 14

3.3 Transformación a QAP 14

3.4 Verificación y solidez 15

4 Conclusiones **16**

A Uso de IA en el desarrollo de este trabajo **17**

1 Introducción

En los últimos años, el aprendizaje automático ha experimentado un crecimiento sin precedentes, tanto en capacidad como en aplicaciones. Modelos cada vez más complejos, como los grandes modelos de lenguaje, se integran en aplicaciones críticas, como el diagnóstico médico [1], la conducción autónoma [2] o la toma de decisiones financieras [3]. Junto a ello, surge un modelo de negocio en el que un proveedor despliega un modelo entrenado y ofrece inferencia a usuarios, normalmente, sin revelar los detalles internos del modelo; es el caso de *Google* con su modelo *Gemini*, *OpenAI* con *ChatGPT*, y otros. Esto introduce un problema de confianza. El usuario no tiene forma de verificar que el proveedor está ejecutando el modelo que afirma ejecutar, ni que lo hace de forma correcta. Por otro lado, el proveedor tiene incentivos económicos para no revelar los pesos de su modelo, pues son su propiedad intelectual y el resultado de un proceso costoso de entrenamiento. De esta forma, surge una tensión entre la necesidad de verificar la inferencia de un modelo y la privacidad del proveedor: ¿es posible que el usuario pueda confiar en el resultado sin que el proveedor tenga que revelar sus secretos?

Las pruebas de conocimiento cero surgieron para permitir a una parte demostrar que ha ejecutado un cómputo correctamente sin revelar información secreta sobre el mismo [4]. Las aplicaciones de estas pruebas son diversas, desde *blockchain* para realizar transacciones y contratos inteligentes privatizando a los usuarios involucrados, hasta en el propio aprendizaje automático para demostrar la ejecución de un modelo determinado sobre unos datos concretos [5].

A pesar de su potencial, la aplicación de pruebas de conocimiento cero a modelos de aprendizaje automático presenta grandes desafíos técnicos [6]. Los protocolos criptográficos operan sobre cuerpos finitos, mientras que los modelos de aprendizaje automático manejan números reales y operaciones no lineales. Unificar estos dos mundos requiere traducir la arquitectura de un modelo y su proceso de inferencia mediante técnicas de aritmetización, cuantización y estrategias específicas para manejar la no linealidad.

En este trabajo se abordarán estos desafíos tanto desde una perspectiva teórica como práctica. En primer lugar, en la Sección 2.1 se presentarán los fundamentos teóricos necesarios para entender el aprendizaje automático y, en concreto, las redes neuronales artificiales, así como el concepto de pruebas de conocimiento cero en la Sección 2.2. A continuación, la Sección 2.3 muestra los obstáculos que surgen al trasladar la inferencia de una red neuronal a un circuito aritmético compatible con un sistema de prueba de conocimiento cero, incluyendo la aritmetización mediante sistemas R1CS, la cuantización de los pesos y las activaciones a aritmética de cuerpo finito y el manejo de funciones de activación no lineales, con un enfoque en la función *ReLU*. En la Sección 3 se presenta una implementación práctica en *MATLAB* como prueba de concepto donde se integran todos los conceptos desarrollados [1]. Finalmente, el Anexo A muestra cómo y con qué fin se ha utilizado la inteligencia artificial durante el desarrollo de este trabajo.

[1] El código LaTeX de esta memoria y la implementación en `MATLAB` se encuentra disponible en el repositorio de *GitHub* `https://github.com/alvaromalfaro/zkml-cripto`.

2 Fundamentos teóricos

2.1 Una aproximación al aprendizaje automático

El **aprendizaje automático** es una rama de la inteligencia artificial centrada en conseguir que los ordenadores simulen la capacidad humana de *aprender*, así como de buscar métodos que les permitan adquirir conocimientos y habilidades para mejorar su rendimiento ante determinadas tareas [7]. A diferencia del aprendizaje humano, que se construye a base de la experiencia adquirida a lo largo de grandes períodos de tiempo, los sistemas de aprendizaje automático, gracias a la potencia de cálculo de los ordenadores, pueden adquirir conocimientos utilizando grandes cantidades de datos en períodos de tiempo relativamente cortos; apenas unos minutos en modelos simples y con conjuntos de datos pequeños, como sería el caso de un *árbol de decisión*, u horas, días o incluso semanas en modelos complejos y con conjuntos de datos masivos, como es el caso de los *grandes modelos de lenguaje*.

2.1.1 Redes neuronales artificiales, aprendizaje e inferencia

Si hay un modelo de aprendizaje automático que ha adquirido notoriedad en las últimas décadas, ese es el de las **redes neuronales artificiales**, incluyendo todas sus variantes. Desde una perspectiva biológica simplificada, el cerebro humano se compone de una gran cantidad de *neuronas* interconectadas entre sí que se comunican mediante *impulsos eléctricos* que reciben a través de sus *dendritas* y transmiten a través de sus *axones* [8]. De manera similar, las redes neuronales artificiales se componen de unidades denominadas *neuronas artificiales*, conectadas entre sí mediante *enlaces* que transmiten señales. Concretamente, estas señales se representan mediante *números reales*, donde cada uno de ellos simboliza la fuerza de la conexión, también conocida como *peso*. Una red neuronal suele contener desde algunas decenas hasta miles o incluso millones de neuronas artificiales, organizadas en diferentes *capas*. En general, las neuronas de cada capa se encuentran conectadas a las de la capa siguiente, aunque hay diferentes tipos de arquitecturas con diferentes configuraciones de capas y conexiones. Cuando una red neuronal contiene dos o más capas ocultas, se denomina *red neuronal profunda*, siendo este el caso mostrado y, en general, el más común en la actualidad.

En una red neuronal artificial, el proceso de **inferencia** se refiere a la generación de una salida a partir de una entrada dada, utilizando los pesos de los enlaces entre las neuronas. En cada neurona, se realiza una suma ponderada de las señales recibidas a través de sus enlaces [9, Capítulo 6]. Así, en una cierta neurona, dado un conjunto de entradas $x_1, x_2, \ldots, x_n$ y un conjunto de pesos $w_1, w_2, \ldots, w_n$ asociados a cada una de las entradas, la entrada total de la neurona se calcula como

$$z = b + \sum_i w_i x_i \tag{1}$$

donde b es el término de sesgo, un valor constante que permite ajustar su umbral de activación [10]. Cuando un peso w_i es positivo, se dice que el enlace es *excitatorio*, mientras que, si es negativo, se dice que es *inhibitorio*.

Tras ello, la neurona aplica una *función de activación* a su entrada total, z, para generar su salida,

$$y = f(z) \tag{2}$$

En general, f suele ser una función no lineal debido a que esto es lo que permite a las redes neuronales aprender patrones complejos en los datos. Hay multitud de funciones de activación, sin embargo, no se profundizará en ellas debido a que no resulta especialmente relevante para el desarrollo del trabajo. Hay extensa literatura sobre el tema, por lo que una simple búsqueda de artículos o libros sobre redes neuronales artificiales o aprendizaje profundo proporcionará una gran cantidad de información al respecto [11, 12]. A pesar de ello, un aspecto interesante a destacar es que es condición necesaria que la función de activación sea diferenciable o, al menos, diferenciable en casi todo punto, ya que esto será necesario en el proceso de entrenamiento, basado en la *propagación hacia atrás* del error y el uso de algoritmos de optimización como el *descenso de gradiente*.

Este proceso de suma ponderada seguido de la aplicación de una función de activación se aplica en cada neurona de la red neuronal, desde la capa de entrada hasta la capa de salida, para generar la salida final de la red; se *infiere* un resultado a partir de la entrada.

Llegados a este punto, se habrá observado que no se ha hecho mención alguna a cómo se determinan los pesos de los enlaces. Esto se debe a que, en general, no se establecen manualmente, más allá de una inicialización aleatoria, sino que se aprenden mediante el proceso de *entrenamiento*. Dado un conjunto de datos de entrenamiento, $\mathcal{X}$, este consiste en cuatro simples pasos [10, Capítulo 3]:

1. Inicializar los pesos de la red neuronal de forma aleatoria.
2. Seleccionar un ejemplo de entrenamiento $x \in \mathcal{X}$ e introducirlo por la capa de entrada, de forma que cada neurona genere una señal de salida que transmitirá a las neuronas de la capa siguiente, y así sucesivamente, hasta llegar a la capa de salida, donde se generará una salida y.
3. Si y es diferente a la salida deseada o esperada, $d(x)$, modificar los pesos de la red neuronal de acuerdo al error.
4. Volver al paso 2 hasta alcanzar un cierto criterio de parada, como puede ser un número máximo de iteraciones o un umbral de error mínimo.

Intuitivamente, es fácil ver que el entrenamiento no es más que un proceso iterativo de prueba y error, donde se ajustan los pesos de la red neuronal para que su salida converja a la salida deseada para cada ejemplo de entrenamiento. Matemáticamente, no es un proceso sencillo. De nuevo, dado que el proceso no resulta relevante para el presente proyecto, una explicación formal y detallada del mismo se puede encontrar en el libro de *Krose y Smagt* [10, Capítulo 4].

2.2 Pruebas de conocimiento cero

Las **pruebas de conocimiento cero** surgen ante la necesidad de demostrar la posesión de una determinada información sin revelar nada sobre ella, solamente la prueba de que se posee [4, Capítulo 4]. Para motivar el concepto, *Alicia* decide registrarse en una página web que le solicita que

demuestre que es mayor de edad. ¿Cómo puede hacerlo revelando *únicamente* su edad? La solución tradicional sería que *Alicia* indique su edad, pero esto genera un problema: ¿cómo puede la página web verificar que *Alicia* es realmente mayor de edad? En los últimos años se ha popularizado solicitar a los usuarios una fotografía de su documento de identidad, de forma que la página web sea capaz de verificar la autenticidad del documento y, por tanto, la edad del usuario. Sin embargo, esto también genera un problema: si el objetivo es demostrar que *Alicia* es mayor de edad, ¿qué necesidad hay de revelar su identidad completa? Es más, ¿existe alguna forma de probar que *Alicia* es mayor de edad sin revelar siquiera su edad?

2.2.1 Pruebas interactivas

Una primera pregunta que puede surgir es: ¿qué es exactamente una **prueba**? En el campo de las matemáticas y, más concretamente, en la *teoría de la demostración*, una prueba es una secuencia de afirmaciones que han sido demostradas como verdaderas, los *axiomas*, o que se derivan de otras afirmaciones verdaderas mediante *reglas de inferencia* [4, Capítulo 4]; en este sentido, una prueba es un objeto matemático que se puede verificar de forma eficiente. En otras áreas, una prueba es un proceso *dinámico* basado en la interacción entre dos partes mediante el cual se establece la veracidad de una afirmación, donde la prueba tiene menor valor que las consecuencias que se derivan de ella.

La interpretación dinámica del concepto de prueba es la más apropiada para pruebas dentro de protocolos criptográficos. En este contexto, una prueba es un *protocolo interactivo* entre dos partes encargadas de llevar a cabo dos tareas computacionales diferentes: producir una prueba y verificarla. Estas partes se denominan **probadora** (P) y **verificadora** (V). La *probadora* es la que posee el conocimiento que se desea demostrar, mientras que la *verificadora* es la que desea verificar la veracidad de la afirmación dada por la probadora. Cada una de las partes se modela mediante una **máquina de Turing interactiva**. Así, la probadora P afirma que una declaración es cierta y el objetivo de la verificadora V es determinar si esa afirmación es veraz mediante un proceso de interrogación a P. Una característica importante de este modelo es que P puede tener un poder computacional ilimitado, mientras que V es una máquina de Turing estrictamente acotada, operando en *tiempo polinomial probabilístico* (TPP). Es decir, V es una máquina de Turing que, además de ser capaz de finalizar su ejecución en un número de pasos acotado por un polinomio del tamaño de la entrada, también es capaz de utilizar aleatoriedad durante su ejecución.

El uso de la aleatoriedad por parte de la verificadora V es crucial para garantizar la seguridad del protocolo. Si esta fuese una máquina completamente determinista, una probadora maliciosa podría predecir qué preguntas formulará V durante el interrogatorio y preparar respuestas específicas para cada una de ellas, permitiéndole engañar a V para que acepte afirmaciones falsas; una verificadora determinista haría que la utilidad del proceso de interacción fuese nula.

Formalmente, un par de máquinas de Turing interactivas (P, V) constituyen un **sistema de prueba interactivo** para un lenguaje L si satisfacen las siguientes condiciones:

- *Completitud*: si la afirmación es verdadera, es decir, $x \in L$, entonces una verificadora honesta

V aceptará la afirmación dada por una probadora honesta P con alta probabilidad, esto es,

$$\Pr[\langle P, V\rangle(x) = 1] \geq \frac{2}{3} \tag{3}$$

- *Solidez*: si la afirmación es falsa, $x \notin L$, entonces ninguna probadora maliciosa B podrá engañar a una verificadora honesta con una probabilidad significativa, es decir,

$$\Pr[\langle B, V\rangle(x) = 1] \leq \frac{1}{3} \tag{4}$$

donde $\langle P, V\rangle(x) = 1$ denota el evento de que V acepte la afirmación dada por P tras ejecutar el protocolo interactivo con entrada x. En otras palabras, un sistema de prueba interactivo para un lenguaje L es un protocolo entre una probadora P y una verificadora V tal que, si la afirmación dada por P es verdadera, V la acepta con alta probabilidad, mientras que, si la afirmación es falsa, V la rechaza con alta probabilidad. Resulta de interés destacar que la probabilidad de error en ambos casos puede ser reducida a valores arbitrariamente pequeños mediante la *repetición* del protocolo un número suficiente de veces.

2.2.2 Propiedad de conocimiento cero

Una vez establecido el concepto de sistema de prueba interativo y por qué es necesario que la verificadora actúe de forma probabilística, el siguiente paso natural es abordar la **propiedad de conocimiento cero**, es decir, demostrar que una afirmación es verdadera sin revelar ningún tipo de información secreta que justifique su veracidad. Así, cabe preguntarse, ¿es posible formalizar matemáticamente la idea de que, durante el proceso de interacción, no se revela ningún tipo de información adicional sobre la probadora? Es necesario introducir el paradigma de la **simulación** [4, Capítulo 4].

El enfoque clásico de la teoría de la información, como el propuesto por *Shannon*, mide la cantidad de información en términos de incertidumbre [13]. Sin embargo, esto no es adecuado para la criptografía moderna, donde el conocimiento se mide en términos de complejidad computacional, es decir, en la cantidad de recursos computacionales necesarios para obtener cierta información. Según *Goldreich*, una prueba no revela ningún conocimiento si todo lo que la verificadora puede calcular tras la interacción con la probadora, también lo podría haber calculado sin interactuar con ella [4, Capítulo 4].

Formalmente, dado un sistema de prueba interactivo (P, V) para un lenguaje L, se conoce como **vista** de una verificadora V^* tras interactuar con una probadora P con entrada x, denotada como $\text{vista}_{V^*}^{P}(x)$ a una variable aleatoria que incluye

1. La entrada pública x.
2. La aleatoriedad interna generada por V^*.
3. Todas las respuestas dadas por P durante la interacción con V^*.

Con ello, se dice que el sistema de prueba interactivo (P, V) tiene la propiedad de *conocimiento*

cero si para cualquier verificadora V^* existe un algoritmo que opere en tiempo polinomial probabilístico M^*, denominado **simulador**, tal que M^* es capaz de generar una transcripción, $M^*(x)$, que es indistinguible de la vista real $\text{vista}_{V^*}^{P}(x)$. Es decir, si la verificadora V^* no puede distinguir entre una conversación real con la probadora P y una conversación ficticia generada por el simulador M^*, entonces la interacción con P no le ha proporcionado ningún conocimiento adicional que no tuviera ya; no se ha revelado ningún conocimiento.

Ahora bien, matemáticamente, ¿qué quiere decir que una cosa sea indistinguible de otra? Esta pregunta permite introducir dos niveles de seguridad en la propiedad de conocimiento cero: **perfecto** y **computacional** [4, Capítulo 4]. El *conocimiento cero perfecto* se refiere a que la transcripción generada por el simulador, $M^*(x)$, sigue una distribución de probabilidad idéntica a la vista real $\text{vista}_{V^*}^{P}(x)$. En cambio, el *conocimiento cero computacional* se produce cuando la transcripción generada por el simulador, $M^*(x)$, es computacionalmente indistinguible de la vista real $\text{vista}_{V^*}^{P}(x)$, es decir, que aunque las distribuciones de probabilidad no sean idénticas, ningún algoritmo limitado a tiempo polinomial probabilístico puede distinguir entre ambos con una probabilidad mayor a un límite despreciable.

Esta definición de conocimiento cero y el paradigma de la simulación proporcionan una base sólida, pero su utilidad práctica depende de los problemas específicos que se pueden resolver mediante pruebas de conocimiento cero. *Goldreich et al.* demostraron que cualquier lenguaje que pueda ser verificado en tiempo polinomial, es decir, cualquier lenguaje en la clase de complejidad NP, posee un sistema de prueba interactivo de conocimiento cero computacional, asumiendo la existencia de funciones de un solo sentido [14]. También, resulta de interés recordar que cualquier problema perteneciente a la clase de complejidad NP se puede reducir a un problema de satisfacibilidad de circuitos booleanos (*CSAT*) en tiempo polinomial por el teorema de *Cook-Levin*, puesto que *CSAT* es un problema NP-completo [15, 16]. A su vez, los circuitos booleanos se pueden simular mediante circuitos aritméticos sobre cuerpos finitos [17, Capítulo 4]. Además, se recuerda, la inferencia en una red neuronal consiste en aplicar una serie de pesos precalculados a unos datos de entrada para obtener una salida, lo cual no deja de ser más que la aplicación de una secuencia de operaciones matemáticas deterministas. Demostrar que se ha ejecutado una red neuronal sobre una entrada específica es, por tanto, una declaración de tipo NP. El problema que surge ahora es, ¿cómo se puede traducir la arquitectura de una red neuronal y su proceso de inferencia a un circuito aritmético?

2.3 Pruebas de conocimiento cero para modelos de aprendizaje automático

Ya se ha establecido que es teóricamente posible demostrar la correcta ejecución de cualquier programa, incluyendo modelos de aprendizaje automático, ocultando sus secretos mediante sistemas de prueba interactivos. Sin embargo, para que esto sea viable en la práctica, el modelo tradicional presenta serias limitaciones, principalmente en términos de eficiencia. Además, la aplicación de pruebas de conocimiento cero a modelos de aprendizaje automático (**ZKML**, por sus siglas en inglés: *Zero-Knowledge Machine Learning*) presenta desafíos de ingeniería adicionales, como la incompatibilidad fundamental entre los tipos de datos y operaciones utilizados en estos modelos y los que se manejan en los protocolos criptográficos.

2.3.1 SNARKs y aritmetización

Para superar las limitaciones de eficiencia, la criptografía moderna ha evolucionado hacia la construcción de **sistemas de prueba no interactivos**, conocidos como **SNARKs** (por sus siglas en inglés: *Succinct Non-interactive Arguments of Knowledge*) [17, Capítulo 1]. Se hace necesario desglosar el acrónimo para entender su significado y las propiedades que se exigirán a un sistema de prueba para modelos de aprendizaje automático. En primer lugar, un sistema de prueba es *sucinto* si el tamaño de la prueba generada por la probadora P es pequeño y el tiempo que tarda la verificadora V en comprobarla es del orden de milisegundos, independientemente del tamaño y complejidad del programa que se está demostrando. Es fácil ver que esta propiedad es clave para la escalabilidad. En segundo lugar, un sistema de prueba es *no interactivo* si no hay un interrogatorio de ida y vuelta; la probadora genera un único mensaje de prueba y lo envía a la verificadora. Para lograr esto, se suele hacer uso de la *heurística de Fiat-Shamir*, que emplea funciones *hash* criptográficas para generar desafíos pseudoaleatorios de forma no interactiva [18]. Por otra parte, existe una distinción técnica entre *argumento* y *prueba*; la prueba tiene solidez estadística, protegiendo contra probadoras maliciosas que puedan tener poder de cómputo ilimitado, mientras que el argumento solo tiene solidez computacional, es decir, un *SNARK* asume que la probadora maliciosa está limitada a tiempo polinomial [17, Capítulo 1]. Finalmente, *conocimiento* se refiere a que no solo debe garantizarse que la afirmación es cierta, sino también que la probadora posee la información o testigo que hace la afirmación cierta.

Cuando a un *SNARK* se le exige ocultar el *testigo*, es decir, la información secreta que justifica la veracidad de la afirmación, se le denomina **zk-SNARK** [17, Capítulo 1]. Como se puede observar, esto es la propiedad de *conocimiento cero* introducida en la Sección 2.2.2. Así, para construir un *zk-SNARK* de un modelo de aprendizaje automático, el primer inconveniente que surge es que un protocolo criptográfico no entiende de neuronas, capas o pesos; operan evaluando polinomios sobre cuerpos finitos $\mathbb{F}_p$, donde p es un número primo criptográficamente grande. Por tanto, es necesario traducir la aquitectura de la red neuronal a un conjunto de restricciones polinomiales mediante un proceso de **aritmetización**.

El estándar para representar circuitos aritméticos es el **Sistema de restricciones de Rango 1** (*R1CS*, por sus siglas en inglés: *Rank-1 Constraint System*) [17, Capítulo 8]. En un sistema R1CS, el cálculo completo de un programa se representa mediante un conjunto de restricciones polinomiales en forma de sistema de matrices, tal que

$$(A \cdot z) \circ (B \cdot z) = C \cdot z \tag{5}$$

donde A, B y C son matrices de coeficientes que codifican la *topología* del circuito matemático, con dimensiones $m \times n$, donde m es el número de restricciones (puertas lógicas o aritméticas) y n es la longitud del vector de variables z. Este vector, denominado *testigo*, contiene todas las variables del sistema, esto es, las entradas públicas (los datos), las entradas privadas (los pesos), todas las variables intermedias generadas durante el cálculo y la constante 1 en la primera posición. Finalmente, el símbolo $\circ$ denota el producto de *Hadamard*, es decir, la multiplicación elemento a elemento de los vectores resultantes.

Para ilustrar cómo se aplica este sistema matricial a una operación fundamental del aprendizaje automático, se considera el cálculo de una única neurona lineal simple, donde una entrada x se multiplica por un peso w para generar una salida y, es decir, se debe probar que $y = w \cdot x$. El primer paso es definir el vector testigo z con todas las variables involucradas junto con la constante, de forma que $z = [1, y, x, w]^\intercal$. A continuación, se deben establecer las matrices A, B y C con el objetivo de aislar los operandos correspondientes. Para la ecuación propuesta, se establecen como sigue:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$$
$$B = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$
$$C = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$$

Con ello, al evaluar el sistema, se obtiene $A \cdot z = x$, $B \cdot z = w$ y $C \cdot z = y$, de modo que la Ecuación 5 se evalúa exactamente como la restricción que se desea probar, es decir, $x \cdot w = y$. La inferencia de una red neuronal completa, con múltiples capas y miles o millones de neuronas, no es más que la concatenación de miles de estas restricciones, añadiendo nuevas variables intermedias al vector testigo por cada operación elemental realizada.

Una vez construido el sistema R1CS, los protocolos modernos interpolan estas matrices para generar un conjunto de polinomios en un formato conocido como **Programas Aritméticos Cuadráticos** (*QAP*, por sus siglas en inglés: *Quadratic Arithmetic Programs*) [19]. Esta transformación es el paso final para que un sistema *zk-SNARK* pueda generar pruebas sucintas. El objetivo de *QAP* es condensar el sistema de restricciones en un conjunto de polinomios utilizando técnicas de interpolación polinómica, normalmente interpolación de *Lagrange*, de forma que la validez de todo el circuito se reduce a verificar una única identidad polinómica:

$$A(x) \cdot B(x) - C(x) = H(x) \cdot Z(x) \tag{6}$$

donde $A(x)$, $B(x)$ y $C(x)$ son polinomios que representan los operandos izquierdo, derecho y la salida en el vector testigo, respectivamente. $Z(x)$ es un polinomio que se anula en las raíces correspondientes a cada puerta lógica y $H(x)$ es el polinomio cociente generado por la probadora para demostrar que $A(x) \cdot B(x) - C(x)$ es divisible por $Z(x)$. Si la probadora ha ejecutado la red neuronal correctamente, entonces la identidad polinómica se cumple y la inferencia es válida [2].

Al reducir todas las operaciones de la red neuronal a la verificación de la divisibilidad de un polinomio, la verificadora puede comprobar la validez de la prueba evaluando los polinomios en un único punto aleatorio, gracias al Lema de *Schwartz-Zippel* [17, Capítulo 4].

2.3.2 Compromisos polinómicos y *blinding*

La aritmetización reduce la verificación de un cálculo a la comprobación de una identidad polinómica. Sin embargo, una verificación directa de la identidad no preserva el conocimiento cero, ya que los coeficientes de los polinomios $A(x)$, $B(x)$ y $C(x)$ son combinaciones lineales de las varia-

[2] La notación $A(x)$, $B(x)$ y $C(x)$ proviene del artículo de *Vitalik Buterin* [20], creador de *Ethereum*, y se han escogido por equivalencia didáctica con las matrices del sistema R1CS.

bles del vector testigo, por lo que su revelación expone la información que se desea proteger. Para que un *SNARK* alcance la propiedad de conocimiento cero, es necesario un esquema de compromiso polinómico y un mecanismo de aleatorización, conocido como *blinding*.

Un **esquema de compromiso polinómico** (PCS, por sus siglas en inglés: *Polynomial Commitment Scheme*) es una construcción criptográfica que permite a una parte comprometerse al valor de un polinomio $f(x) \in \mathbb{F}_p[x]$ produciendo un valor corto denominado *compromiso*, normalmente un elemento de un grupo criptográfico, y abrir posteriormente el compromiso en cualquier punto r revelando $f(r)$ junto con una prueba π de que dicha evaluación es consistente con el polinomio comprometido [17, Capítulo 7]. Un PCS debe cumplir dos propiedades:

- **Vinculación** (*binding*): una vez publicado el compromiso, la probadora no puede abrir el compromiso a un valor distinto de $f(r)$ sin romper una asunción criptográfica subyacente.
- **Ocultación** (*hiding*): el compromiso no revela información computacionalmente útil sobre los coeficientes de f.

Con un PCS, el protocolo se modifica de forma que la probadora envía Com(A), Com(B), Com(C) y Com(H) en lugar de los polinomios. La verificadora, mediante *Fiat-Shamir*, deriva un punto r y solicita las *aperturas* en él. Así, la igualdad polinómica se comprueba sobre los valores abiertos; la verificadora nunca accede a los coeficientes de los polinomios.

Ahora bien, las evaluaciones de $A(r)$, $B(r)$ y $C(r)$ siguen siendo combinaciones lineales del vector testigo, por lo que un verificador malicioso que observe múltiples evaluaciones podría plantear un sistema lineal y recuperar parte del testigo. Para evitarlo, se introduce el **blinding**, en el que la probadora añade términos aleatorios a los polinomios antes de comprometerse a ellos, de forma que las evaluaciones fuera de los puntos de interpolación quedan aleatorizadas. Distintos *SNARKs* emplean diferentes técnicas de *blinding*. Por ejemplo, *Groth* [21] aleatoriza la propia prueba añadiendo términos aleatorios a los elementos A, B, C que la probadora envía. Otros sistemas, como *PLONK* [22] aleatorizan directamente los polinomios del circuito añadiendo múltiplos del polinomio de anulación $Z(x)$. En cualquier caso, el resultado es el mismo, las distribuciones de las pruebas reales y simuladas sean indistinguibles, garantizando así la propiedad de conocimiento cero.

2.3.3 Cuantización y aritmética en $\mathbb{F}_p$

Tras establecer el proceso de aritmetización, ahora es necesario abordar el desafío de la incompatibilidad entre los tipos de datos y operaciones utilizados en los modelos de aprendizaje automático y los que se manejan en los protocolos criptográficos.

Como se mencionó en la Sección 2.1.1, los modelos de aprendizaje automático, y en particular las redes neuronales, operan con números reales utilizando representaciones de punto flotante, típicamente de 16 o 32 bits. En cambio, un sistema de pruebas como *zk-SNARK* evalúa circuitos aritméticos sobre cuerpos finitos $\mathbb{F}_p$, donde p es un número primo grande. En este contexto, los únicos valores válidos son los enteros positivos en el rango $[0, p-1]$. Para llevar a cabo la inferencia de una red neuronal dentro de un circuito *zk-SNARK*, será necesario transformar los pesos y activaciones de la red

neuronal a este dominio matemático, lo cual se puede lograr mediante un proceso de **cuantización** [23].

El primer paso consiste en transformar los números decimales a enteros mediante aritmética de punto fijo. Para ello, se define un factor de escala S y, con ello, un valor real r se cuantiza a su representación entera r_q como:

$$r_q = Q(r) = \text{Int}(r/S) + Z \tag{7}$$

donde Q es un operador de cuantización, Int es la función de redondeo y truncamiento, y Z es un entero de desplazamiento que representa el valor cero en la representación cuantizada.

Un aspecto importante a destacar es que los pesos de una red neuronal pueden ser positivos y negativos, como se indicó en la Sección 2.1.1. Sin embargo, en un sistema de pruebas como *zk-SNARK*, solo se pueden manejar enteros positivos, puesto que el cuerpo finito $\mathbb{F}_p$ no incluye números negativos. Para representar números negativos, se puede utilizar el inverso aditivo módulo p, de forma que un valor $-x$ se representa como $p-x \in \mathbb{F}_p$. Esto es consistente con aritmética de cuerpos finitos, ya que $x+(p-x) \equiv 0 \pmod{p}$. Para distinguir entre números positivos y negativos, se puede establecer un umbral de cuantización, de forma que los elementos en el rango $[0, p/2]$ se interpretan como positivos, mientras que los elementos en el rango $[p/2, p-1]$ representan valores negativos con magnitud $p-x$.

A pesar de ello, esta representación introduce un coste adicional en el circuito aritmético. Algunas funciones de activación, como *ReLU*, requieren determinar el signo de un valor, por lo que deben descomponer el elemento en su representación binaria para verificar si se encuentran por encima o por debajo del umbral de cuantización, generando restricciones adicionales en el circuito. *Feng et al.* proponen optimizaciones como la agrupación de bits de signo, que reduce el número de restricciones necesarias generadas por estas verificaciones [24]. Y, ahora que se menciona la función de activación *ReLU*, es buen momento para enfrentar un último desafío.

2.3.4 Lidiando con la no linealidad

La aritmetización de operaciones lineales de una red neuronal, como la mostrada en la Ecuación 1, se traduce de forma directa a un sistema R1CS, como se ha ilustrado en la Sección 2.3.1. Sin embargo, como se indicó en la Sección 2.1.1, es común que las redes neuronales apliquen funciones de activación no lineales, que no se pueden expresar como una simple multiplicación entre dos combinaciones lineales. Se presenta el caso de la función de activación *ReLU* (*Rectified Linear Unit*), que se define como

$$\text{ReLU}(x) = \max(0, x) \tag{8}$$

Es fácil ver que su principal problema es que requiere determinar el signo de su entrada, es decir, si x es positivo, el resultado es x, mientras que, si x es negativo, el resultado es 0. Realizar esta comprobación no es trivial, ya que la comparación de magnitudes no es una operación nativa en $\mathbb{F}_p$. Para resolver este problema, se puede utilizar la técnica de **descomposición en bits** [17, Capítulo 6]. Dado un elemento $x \in \mathbb{F}_p$, se introducen $n = \log_2(p)$ variables auxiliares $b_0, b_1, \ldots, b_{n-1}$ en el

vector testigo, que representan los bits de x. Para que esta descomposición sea válida, se imponen dos restricciones:

1. Cada variable b_i debe ser un valor booleano, lo cual se puede garantizar mediante la restricción $b_i \cdot (b_i - 1) = 0$, que solo se cumple si b_i es 0 o 1.

2. Los bits deben permitir reconstruir el valor original x, esto es,

$$x = \sum_{i=0}^{n-1} b_i \cdot 2^i \tag{9}$$

Una vez descompuesto el elemento x en sus bits, se puede determinar su signo verificando el bit más significativo, de forma que si $b_{n-1} = 0$, entonces $x < p/2$ y se interpreta como positivo, mientras que si $b_{n-1} = 1$, entonces $x \geq p/2$ y se interpreta como negativo. Con ello, es trivial implementar la función de activación *ReLU* mediante la restricción [3]

$$\text{ReLU}(x) = x \cdot s \tag{10}$$

donde s es una variable binaria derivada del bit de signo.

Sin embargo, el coste del procedimiento es elevado. La restricción $b_i \cdot (b_i - 1) = 0$ genera una restricción R1CS por cada bit, generando un total de $n = \log_2(p)$ restricciones por cada función de activación *ReLU* aplicada. Dado que una red neuronal típica puede contener miles o millones de neuronas, cada una aplicando una función de activación no lineal, el número total de restricciones generadas puede ser muy elevado, afectando a la eficiencia del sistema de prueba. De nuevo, *Feng et al.* proponen optimizar mediante la agrupación de bits de signo [24]. Asimismo, *Sun et al.* formalizan que el enfoque de descomposición en bits tiene un coste $\Omega(mnlog_2B)$ para una capa de dimensiones $m \times n$ con valores acotados por B [25].

Una alternativa a la descomposición en bits es utilizar **tablas de búsqueda**, que permiten verificar operaciones no lineales precalculando una tabla de valores válidos y comprobando la pertenencia del resultado a la tabla [26]. Esto reduce significativamente el número de restricciones necesarias.

En general, los procesos de aritmetización, cuantización y manejo de la no linealidad conllevan un coste adicional significativo respecto a la inferencia directa. Trabajos como el de *Feng et al.* han demostrado que la generación de una prueba en una red neuronal puede ser de varios órdenes de magnitud más lenta que la inferencia original, según la arquitectura y el sistema de prueba utilizado [24]. Esto hace que la corrección y privacidad en el campo de *ZKML* se obtenga a cambio de un coste computacional elevado, motivando la investigación en optimizaciones y nuevas técnicas que busquen reducirlo.

2.3.5 *Frameworks* y herramientas modernas

Los fundamentos teóricos presentados en las secciones anteriores han dado lugar, en los últimos años, a la aparición de múltiples *frameworks* y herramientas que implementan pruebas de conoci-

[3] Notación escogida como simplificación didáctica. En la práctica, los *frameworks* suelen llevar a cabo implementaciones más complejas.

miento cero para modelos de aprendizaje automático. A continuación, se presentan brevemente algunos de los más relevantes.

Por un lado, **EZKL** es una herramienta de código abierto diseñada para generar pruebas de inferencia verificable a partir de modelos exportados en formato *ONNX* (*Open Neural Network Exchange*) [27]. Internamente, EZKL traduce la arquitectura de la red neuronal a un circuito aritmético compatible con el sistema de prueba **Halo2**, que utiliza un esquema basado en el paradigma **PLONK** (*Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*) [22], lo que permite prescindir de una ceremonia de configuración de confianza requerida por algunos sistemas. Además, incorpora soporte nativo para tablas de búsqueda, facilitando la gestión eficiente de funciones de activación no lineales.

Por otra parte, **Circom** [28], tecnología de sello español, es un lenguaje de programación de circuitos aritméticos R1CS, ampliamente utilizado en el ecosistema *blockchain*, especialmente en aplicaciones sobre *Ethereum*. A diferencia de EZKL, Circom no se especializa en modelos de aprendizaje automático, sino que proporciona un entorno general para construir circuitos aritméticos de forma modular; su aplicación a modelos de aprendizaje automático requiere, por tanto, de una traducción manual de la arquitectura del modelo o mediante librerías adicionales.

Luego, **ZEN**, propuesto por *Feng et al.* [24], es un compilador optimizado específicamente para la generación de pruebas de inferencia en redes neuronales. Su contribución principal reside en una serie de optimizaciones a nivel de circuito, que reducen el número de restricciones generadas por las operaciones de cuantización y las funciones de activación no lineales.

Finalmente, **ZKLLM**, propuesto por *Sun et al.* [25], representa la "frontera" actual del campo, pues aborda la generación de pruebas de conocimiento cero para grandes modelos de lenguaje. Los autores proponen técnicas como *tlookup*, una generalización de las tablas de búsqueda a tensores, para manejar operaciones no lineales presentes en arquitecturas basadas en *Transformers*, la base de los grandes modelos de lenguaje, para poder escalar así a modelos masivos.

A pesar de los avances logrados por estas herramientas, su complejidad interna dificulta la comprensión de los mecanismos subyacentes. Para ilustrar el proceso completo de forma transparente y didáctica, se presenta, en la siguiente sección, una implementación simple en `MATLAB` que reproduce las etapas del proceso de inferencia verificable, aunque sin alcanzar la complejidad de los *frameworks* modernos. Se llevará a cabo la cuantización de los pesos y activaciones, la aritmetización de la red neuronal mediante un sistema R1CS y la generación y verificación de la prueba.

3 Prueba de concepto en *MATLAB*

Tras haber establecido los fundamentos teóricos del problema y revisar algunos *frameworks* modernos, en esta sección se presenta una implementación práctica en `MATLAB` que reproduce, de forma simplificada, el proceso completo de inferencia verificable sobre una red neuronal. El objetivo, al final, es adquirir una comprensión más profunda de los mecanismos subyacentes.

Es importante destacar que la implementación tiene un propósito ilustrativo y didáctico. Su objetivo es reproducir el procedimiento que conecta la inferencia de una red neuronal con su verificación

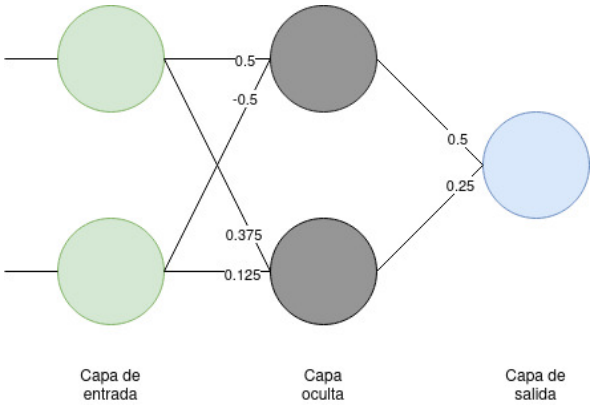

Figura 1: Arquitectura de la red neuronal utilizada en la implementación.

mediante una identidad polinómica: cuantización, aritmetización en R1CS, transformación a QAP, generación del polinomio cociente y verificación por evaluación en un punto aleatorio. Ahora bien, los componentes que dotan la propiedad de conocimiento cero, descritos en la Sección 2.3.2, quedan fuera del alcance de esta implementación. Su implementación requiere primitivas criptográficas, grupos de orden primo grande y, en determinados casos, emparejamientos bilineales sobre curvas elípticas. Por tanto, el sistema implementado constituye un *SNARK* sobre la inferencia de la red, pero no un *zk-SNARK*.

La implementación se incluye en un fichero `MATLAB` junto a esta memoria. La red neuronal escogida es deliberadamente pequeña, $2 \times 2 \times 1$ con activación *ReLU* en la capa oculta (Figura 1) y un cuerpo finito también pequeño ($\mathbb{F}_{65537}$), de forma que todos los valores intermedios puedan inspeccionarse manualmente y los polinomios resultantes se manejan sin estructuras de datos especializadas.

3.1 Cuantización y aritmética modular

El primer paso del proceso consiste en transformar todos los valores que intervienen en la inferencia (entradas, pesos y activaciones) al dominio del cuerpo finito $\mathbb{F}_{65537}$. Para ello, se define un factor de escala $S = 16$. La cuantización de un valor real r se realiza de acuerdo con la Ecuación 7, con $Z = 0$ pues no se requiere un desplazamiento y el resultado se reduce módulo 65537 ($r_q = \text{round}(r \cdot S) \pmod{65537}$). Los valores negativos se representan, como se explicó en la Sección 2.3.3, mediante el inverso aditivo módulo 65537. Así, un peso de -0.5 se cuantiza primero a -8 y, luego, se interpreta como $65537 - 8 = 65529 \in \mathbb{F}_{65537}$. En cualquier operación posterior, el módulo mantiene el valor dentro del rango $[0, 65536]$ y la interpretación de positivo o negativo se realiza únicamente al final, cuando se requiere recuperar el signo, por ejemplo, para evaluar la función de activación *ReLU*.

Un aspecto importante a destacar es que, como la aritmetización exige que cada multiplicación entre valores cuantizados se exprese como una restricción R1CS, no se realiza un reescalado intermedio entre capas. Esto implica que las unidades se acumulan a lo largo de la red. Tras la primera capa, los productos se encuentran en unidades de S^2, tras la segunda capa en unidades de S^3. Así, la salida cuantizada y_q obtenida al final de la inferencia satisface $y \approx y_q / S^L$, donde L es el número de capas. Esta acumulación es manejable en este caso, pero en redes neuronales profundas sería necesario introducir operaciones de reescalado que también deben ser aritmetizadas mediante restricciones

R1CS adicionales, añadiendo más complejidad al proceso.

3.2 Aritmetización en R1CS

Una vez la red neuronal reside en el dominio de $\mathbb{F}_{65537}$, la inferencia debe traducirse a un sistema de restricciones R1CS de la forma $A \cdot z \circ B \cdot z = C \cdot z$. El vector testigo z incluye $n = 20$ variables agrupadas en cuatro categorías: la constante 1 en la primera posición, las entradas y la salida públicas x_1, x_2, y, los pesos privados de la red W_1, W_2 y diez variables intermedias correspondientes a productos parciales, *bits* de signo y activaciones tras la función *ReLU*. Luego, para la formulación de las restricciones, la inferencia genera $m = 11$ restricciones que cubren los productos peso-entrada de la primera capa, las dos aplicaciones de *ReLU*, los productos peso-activación de la capa de salida y la suma final. La especificación completa del testigo y las matrices A, B y C se incluyen en el código `MATLAB` adjunto.

Sí resulta interesante examinar una restricción que muestra los dos mecanismos no triviales que surgen al aritmetizar la red neuronal. Por un lado, considerando la restricción correspondiente a la activación de la primera neurona oculta

$$(t_1 + t_2) \cdot sgn_1 = a_{1,1} \tag{11}$$

donde $t_1 = w_{11} \cdot x_1$ y $t_2 = w_{12} \cdot x_2$ son productos calculados en restricciones anteriores, $sgn_1 \in \{0, 1\}$ es una variable que indica si la entrada a la función de activación es positiva o negativa y $a_{1,1}$ es la salida tras aplicar *ReLU*. Esta restricción se codifica colocando dos coeficientes no nulos en la fila $i = 5$ de la matriz A (en las columnas de t_1 y t_2), un coeficiente no nulo en la columna de sgn_1 en B y un coeficiente no nulo en la columna de $a_{1,1}$ en C. De este modo, $A \cdot z$ produce $t_1 + t_2$ sin necesidad de introducir una restricción dedicada para sumar, pues queda realizada por la propia combinación lineal.

El otro mecanismo, la implementación de *ReLU* como producto $s \cdot sgn$ siguiendo la Ecuación 10, requiere una observación importante. Tal como se ha desarrollado en la Sección 2.3.4, garantizar que sgn_1 refleje el signo de la pre-activación requiere una descomposición en bits, lo que añade $\lceil \log_2(65537) \rceil = 17$ restricciones adicionales por activación. Esta implementación omite esa descomposición y se limita a imponer que sgn_1 sea booleano mediante la restricción $sgn_1 \cdot sgn_1 = sgn_1$, lo que supone una simplificación y se confía en que la probadora proporcione el *bit* de signo correcto. Esto sería inadmisible en un sistema real, donde una probadora maliciosa podría declarar $sgn_1 = 1$ para una entrada negativa. Sin embargo, para el propósito de esta implementación y para no saturar el sistema de restricciones, se ha optado por esta simplificación.

3.3 Transformación a QAP

Tras construir el sistema R1CS, verificar las $m = 11$ restricciones requeriría que la verificadora accediese al vector testigo, comprometiendo cualquier intento de conocimiento cero. La transformación a un Programa Aritmético Cuadrático (QAP) condensa el sistema entero en una única identidad polinómica.

La idea parte de asignar los enteros $\{1, 2, \ldots, m\}$ a las m restricciones y se interpretan las columnas de las matrices A, B, C como vectores de valores indexados por esos puntos, la interpolación de *Lagrange* permite construir, para cada columna j, tres polinomios $L_j^A(x)$, $L_j^B(x)$ y $L_j^C(x)$ de grado máximo $m-1$ tales que $L_j^A(i) = A_{ij}$, e igual para B y C. Al combinarlos con el testigo, se obtienen los polinomios $A(x) = \sum_j z_j \cdot L_j^A(x)$, $B(x) = \sum_j z_j \cdot L_j^B(x)$ y $C(x) = \sum_j z_j \cdot L_j^C(x)$, que satisfacen $A(i) = (A \cdot z)_i$ en cada punto de evaluación. Lo mismo se realiza para $B(i)$ y $C(i)$.

Así, si el testigo es válido, $T(x) = A(x) \cdot B(x) - C(x)$ se anula en los m puntos y es divisible por el polinomio de anulación $Z(x) = \prod_{i=1}^{m}(x-i)$, existiendo un cociente $H(x) = T(x)/Z(x)$ que satisface la Ecuación 6, es decir, todas las restricciones se cumplen simultáneamente.

3.4 Verificación y solidez

La verificación de la prueba se reduce a comprobar la identidad de la Ecuación 6. La clave está en aplicar el lema de *Schwartz-Zippel*. Por él, dados dos polinomios distintos p_1 y q_1 de grado máximo d sobre $\mathbb{F}_p$, la probabilidad de que coincidan en un un punto r elegido uniformemente al azar está acotada por d/p. Basta entonces con que la verificadora elija un punto $r \in \mathbb{F}_{65537}$ y compruebe $A(r) \cdot B(r) - C(r) = H(r) \cdot Z(r) \pmod{65537}$. Con $d \leq 20$ y $p = 65537$, la cota es aproximadamente 0.0003, en un sistema real, con p grande, hace esta probabilidad despreciable.

Al seleccionar r, se han excluido los puntos de interpolación $\{1, 2, \ldots, m\}$, pues en ellos $Z(r) = 0$ y se cumple la igualdad para cualquier $H(r)$. Además, en un sistema real, el punto r no se escoge mediante un generador de números aleatorios, sino que se deriva mediante la heurística de *Fiat-Shamir*, aplicando una función *hash* sobre los compromisos polinómicos del protocolo.

Finalmente, para verificar la propiedad de solidez, se muestra como una probadora maliciosa "altera" los pesos del modelo, w_{11} en concreto, tras publicar la prueba honesta e intentar reutilizar el polinomio $H(x)$ original. Al recalcular $A(r)$ con el testigo alterado, la igualdad deja de cumplirse y la verificadora rechaza la prueba. Con ello, aunque la implementación no incluye el mecanismo de conocimiento cero, sí satisface las propiedades de completitud y solidez computacional, constituyendo un SNARK sobre la inferencia de la red neuronal sobre el cual, un sistema completo, añadiría los compromisos polinómicos y el *blinding* para alcanzar el conocimiento cero.

4 Conclusiones

A Uso de IA en el desarrollo de este trabajo

Durante la realización del trabajo, se ha utilizado la inteligencia artificial para diferentes propósitos que se detallan a continuación:

- **Búsqueda y revisión bibliográfica**: se ha empleado la herramienta *NotebookLM* de Google para realizar una búsqueda inicial de información sobre el campo de *ZKML* y un filtrado preliminar de artículos. Posteriormente, utilizando la función de preguntas y respuestas, se llevaron a cabo algunas consultas específicas sobre conceptos técnicos que no estaban claros, facilitando la comprensión de ciertos aspectos teóricos.

- **Estructuración**: una vez recopilada la información y definidos los aspectos a tratar, se utilizó, también, *NotebookLM* para generar la estructura de contenidos de la memoria, buscando organizar los temas de forma lógica.

- **Plantilla LaTeX**: la plantilla de LaTeX utilizada para esta memoria se obtuvo a mediante *Claude*, que generó un ejemplo de plantilla académica a partir de la estructura definida, indicando que se trataba de un trabajo para una asignatura y solicitando que incluyese la identidad visual de la universidad.

- **Implementación**: durante la implementación en `MATLAB`, se empleó *Claude Code* para organizar la implementación, resolver dudas puntuales y durante la transformación de la aritmetización a QAP, donde se le solicitó generar las funciones de división de polinomios, la interpolación de *Lagrange* y la evaluación de polinomios en un punto dado sobre cuerpos finitos.

Referencias

[1] S. Maity and M. J. Saikia, “Large language models in healthcare and medical applications: A review,” *Bioengineering*, vol. 12, no. 6, 2025.

[2] J. Li, J. Li, G. Yang, L. Yang, H. Chi, and L. Yang, “Applications of large language models and multimodal large models in autonomous driving: A comprehensive review,” *Drones*, vol. 9, no. 4, 2025.

[3] M. Darwish, E. E. Hassanien, and A. H. B. Eissa, “Stock market forecasting: From traditional predictive models to large language models,” *Computational Economics*, Jun 2025.

[4] O. Goldreich, *Foundations of Cryptography, Volume 1, Basic Tools*. Cambridge: Cambridge University Press, 2001.

[5] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, “A survey on the applications of zero-knowledge proofs,” 2026.

[6] Z. Peng, C. Zhao, T. Wang, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, and S. Zhang, “A survey of zero-knowledge proof based verifiable machine learning,” 2026.

[7] H. Wang, C. Ma, and L. Zhou, “A Brief Review of Machine Learning and Its Application,” in *2009 International Conference on Information Engineering and Computer Science*, pp. 1–4, 2009.

[8] M. Islam, G. Chen, and S. Jin, “An Overview of Neural Network,” *American Journal of Neural Networks and Applications*, vol. 5, p. 05, 06 2019.

[9] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed., 2026. Online manuscript released January 6, 2026.

[10] B. Kröse and P. van der Smagt, *An Introduction to Neural Networks*. The University of Amsterdam, eighth ed., November 1996.

[11] V. Kunc and J. Kléma, “Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks,” 2024.

[12] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, “Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark,” 2022.

[13] C. E. Shannon, “A Mathematical Theory of Communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 623–656, 1948.

[14] O. Goldreich, S. Micali, and A. Wigderson, “Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems,” *J. ACM*, vol. 38, pp. 690–728, July 1991.

[15] S. A. Cook, “The complexity of theorem-proving procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC ’71, (New York, NY, USA), pp. 151–158, Association for Computing Machinery, 1971.

[16] L. A. Levin, “Universal Sequential Search Problems,” *Problems of Information Transmission*, vol. 9, no. 3, pp. 265–266, 1973. Original Russian article in *Problemy Peredachi Informatsii*, 9(3):115–116, 1973.

[17] J. Thaler, “Proofs, Arguments, and Zero-Knowledge,” *Foundations and Trends in Privacy and Security*, vol. 4, pp. 117–660, 12 2022.

[18] A. Fiat and A. Shamir, “How to prove yourself: practical solutions to identification and signature problems,” in *Proceedings on Advances in Cryptology—CRYPTO ’86*, (Berlin, Heidelberg), pp. 186–194, Springer-Verlag, 1987.

[19] R. Gennaro, C. Gentry, B. Parno, and M. Raykova, “Quadratic Span Programs and Succinct NIZKs without PCPs,” in *Advances in Cryptology – EUROCRYPT 2013* (T. Johansson and P. Q. Nguyen, eds.), (Berlin, Heidelberg), pp. 626–645, Springer Berlin Heidelberg, 2013.

[20] V. Buterin, “Quadratic Arithmetic Programs: from Zero to Hero.” https://medium.com/@VitalikButerin/quadratic-arithmetic-programs-from-zero-to-hero-f6d558cea649, 2016. Medium. Accedido: 15 de abril de 2026.

[21] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, vol. 9666 of *Lecture Notes in Computer Science*, pp. 305–326, Springer, 2016.

[22] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge.” Cryptology ePrint Archive, Paper 2019/953, 2019.

[23] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A Survey of Quantization Methods for Efficient Neural Network Inference,” 2021.

[24] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu, “ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences.” Cryptology ePrint Archive, Paper 2021/087, 2021.

[25] H. Sun, J. Li, and H. Zhang, “zkLLM: Zero Knowledge Proofs for Large Language Models,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS ’24, (New York, NY, USA), pp. 4405–4419, Association for Computing Machinery, 2024.

[26] A. Gabizon and Z. J. Williamson, “plookup: A simplified polynomial protocol for lookup tables.” Cryptology ePrint Archive, Paper 2020/315, 2020.

[27] The EZKL Authors, “Ezkl documentation: Verifiable machine learning using zero-knowledge proofs.” https://docs.ezkl.xyz/, 2024. Accessed: 2026-04-20.

[28] M. Belles-Munoz, M. Isabel, J. L. Munoz-Tapia, A. Rubio, and J. Baylina, “ Circom: A Circuit Description Language for Building Zero-Knowledge Applications ,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, pp. 4733–4751, Nov. 2023.